

Managed Policy Delivery Pipeline for Island Using OpenTofu and Terraform

Executive Summary

You are inheriting a partially manual workflow where another team performs policy research and updates, and Terraform runs are executed in a sandbox and then “propagated” to production. The target state is an end-to-end managed pipeline that turns policy changes into **versioned, testable, reviewable, and auditable** deliveries—reducing operational risk (accidental production changes, inconsistent environments, drift, and secret leakage) while improving throughput and compliance evidence.

A rigorous design hinges on four “control planes” that should be made explicit during discovery:

Change control (how policy intent is proposed, reviewed, and approved), **execution control** (where plan/apply runs happen and under which identity), **state control** (how desired vs actual state is tracked and protected), and **verification & monitoring** (how correctness and drift are detected and acted on). Terraform’s own documentation stresses that secrets placed in configuration can end up in **state and plan files**, making state governance and artifact handling foundational.

Because major unknowns remain (repo layout, CI/CD platform, cloud provider(s), and whether an Island Terraform provider exists), the recommended approach is to implement the work in **phases**: (1) discovery and inventory with a faithful mapping of the current manual flow, (2) “plan-only” PR checks + policy-as-code validation to tighten governance without changing production behavior, (3) automated sandbox apply + verification, and (4) gated promotion to production with drift detection and observability. This sequencing aligns with proven PR-based automation approaches and avoids premature state/toolchain migrations.

Context and Assumptions

Explicit assumptions to validate during discovery

Island API and policy surface

- Island policy updates are ultimately applied via either (a) an API and/or (b) an admin console. A public UK Digital Marketplace listing claims “Island API supports ... create, read, update and delete policy, policy elements (such as URL lists) ... and ... export logs,” implying an automation-ready interface even if internal/official API docs are gated.
- Island’s policy model likely has separations such as **policy**, **policy elements** (e.g., URL lists), and **assignments/targets** (e.g., users/groups/devices). This structure is not confirmed; treat as a hypothesis until you inspect the actual policy framework and API schema.
- Island emphasizes backward compatibility concerns in its API engineering discussions, which matters for automation stability and versioning expectations.

IaC toolchain and execution

- Terraform is currently the primary runner. You may introduce OpenTofu either as a full migration or as a coexistence period with one “apply authority” per scope to avoid state mutation conflicts. OpenTofu publishes Terraform-to-OpenTofu migration guides that emphasize disaster recovery planning, applying all pending Terraform changes first, backing up state/code, then running `tofu init`, `tofu plan`, and only then `tofu apply`.
- Provider sourcing must be made explicit: Terraform defaults provider hostnames to `registry.terraform.io` if omitted, which affects reproducibility and compatibility during migration/coexistence.
- Both Terraform and OpenTofu rely on a dependency lock file (`.terraform.lock.hcl`) to pin provider versions, include checksums, and support code review of dependency changes; Terraform explicitly recommends committing it.

State, secrets, and artifacts

- State/backends are treated as security-critical. Terraform notes that state and plan files can include sensitive values and recommends securing them accordingly (access control, encryption, and careful handling).
- JSON exports of plans/states are high-risk artifacts: OpenTofu warns that `tofu show -json` returns sensitive values in plaintext.

CI/CD controls

- You will likely implement factorized approvals at the CI environment level. GitHub environments support deployment protection rules (manual approvals, delays, branch restrictions) that block jobs targeting protected environments until requirements are met.
- GitLab supports protected environments and deployment approvals that block deployments until required approvals are granted.
- Short-lived federation (OIDC) is preferred over long-lived secrets where possible: GitHub documents using OIDC to access cloud resources without storing long-lived credentials as secrets.

Discovery and Inventory Process

A good discovery process does two things at once: it inventories artifacts **and** it captures why today’s manual process exists (implicit controls, heuristics, and institutional knowledge). Terraform’s validation/testing features can help you later encode manual acceptance criteria into checks and tests, but only after you understand what those criteria are.

Stepwise discovery process

Map the manual workflow first (before repo spelunking).

Interview the current team and capture, at minimum: triggers (what prompts research), inputs (sources of truth), decision rules (how they choose allow/deny patterns, exceptions), update mechanics (where policies are edited), and verification (how they decide sandbox is “good”). This becomes the baseline you will align to and gradually codify into automation.

Inventory repositories and root modules.

Identify every Terraform/OpenTofu “root” (directories independently initialized/applied). This matters because PR automation tools like Atlantis detect Terraform projects and run `plan` against changed directories.

Inventory backends, state locations, and locking.

Document remote backend types, exact bucket/container paths, locking mechanisms, and access controls. Terraform notes that backends store state and provide APIs for locking (if supported), and state locking protects against concurrent writers corrupting state.

Inventory workspaces and environment isolation patterns.

Workspaces can be useful but are explicitly not appropriate for deployments requiring separate credentials and access controls; this strongly influences how you model sandbox/staging/prod.

Inventory providers, source addresses, and lock files.

Record the `required_providers` entries and whether provider hostnames are explicit. Terraform’s provider registry protocol docs clarify that omitted hostnames default to `registry.terraform.io`.

Verify `terraform.lock.hcl` usage and whether it is committed; Terraform recommends committing it to review dependency changes.

Inventory policy artifacts.

Find where “policy intent” lives: HCL resources, JSON/YAML lists, templates, or scripts. Also capture how policy is mapped to environments/tenants (IDs, selectors, naming conventions). The UK listing suggests Island policy elements like URL lists exist, which frequently become separate managed entities.

Inventory tests, validation gates, and promotion steps.

Find existing `terraform test`/OpenTofu tests, OPA/Conftest policy checks, scanners, and any manual checklists. Terraform warns tests can create real infrastructure (and cost money), which informs how you scope integration tests.

Discovery checklist table

Area	What to capture	Why it matters	Primary reference anchors
Repos & endpoints	All repos; root module dirs; module dirs; ownership	Needed to target PR plans/applies correctly	Atlantis autoplan patterns for multi-dir repos
OpenTofu vs Terraform usage	Binary usage per env; pinned versions; local vs CI runs	Controls migration/coexistence risk	OpenTofu migration steps emphasize DR/backup/clean plan state
Backends & state	Backend type/config; encryption; access; state paths; versioning	State is sensitive and operationally critical	Terraform: backends store state/locking; sensitive data in state/plan
Locking	Lock mechanism; lock failures; unlock procedures	Prevent concurrent state writers	Terraform state locking guidance
Workspaces	Workspace usage; mapping to env/tenant; rationale	Workspaces can cause isolation pitfalls	Terraform warns about workspaces for separate creds/access
Providers & versions	<code>required_providers</code> source addresses; lockfile; upgrade cadence	Reproducibility and migration safety	Terraform defaults provider hostname; lockfile behavior

Area	What to capture	Why it matters	Primary reference anchors
Policy files & data	Where policy intent lives (HCL/JSON/YAML); IDs; targets	Defines what “policy update” means in code	Island API claims policy CRUD + URL lists
Validation & tests	terraform/tofu validate/test; OPA/Conftest; scanners	Prevent unsafe changes before apply	Terraform validate and test docs
Promotion workflow	Sandbox apply; prod promotion steps; approvals; rollback	This is the “pipeline to-be” blueprint	GitHub/GitLab environment approvals for deploy gating
Drift & monitoring	Drift checks; log exports; SIEM ingestion	Drift detection and audit evidence	Island log export claim; HCP drift detection capabilities

Starter inventory script

This script produces a first-pass inventory of Terraform/OpenTofu roots, backends, lock/test files, and policy-ish keywords. Customize keywords once you learn Island’s actual policy terminology.

```
bash
Copy
```

```
#!/usr/bin/env bash
set -euo pipefail

ROOT="${1:-.}"
OUT="${2:-iac_policy_inventory_$(date +%Y%m%d_%H%M%S).txt}"

echo "Scanning repo: $ROOT" | tee "$OUT"
echo "" | tee -a "$OUT"

echo "== Root candidates (dirs with terraform/tofu blocks) ==" | tee -a "$OUT"
find "$ROOT" -type f \( -name "*.tf" -o -name "*.tofu" \) -print0 \
| xargs -0 -n1 dirname \
| sort -u \
| while read -r d; do
    if rg -q '\s*terraform\s*\{' "$d"/*.tf "$d"/*.tofu 2>/dev/null; then
        echo "ROOT: $d" | tee -a "$OUT"
    fi
done

echo "" | tee -a "$OUT"
echo "== Backend blocks ==" | tee -a "$OUT"
```

```
rg -n "backend\s+" "[^"]+" "$ROOT" | tee -a "$OUT" || true
```

```
echo "" | tee -a "$OUT"
```

```
echo "== Cloud/remote blocks (Terraform/HCP) ==" | tee -a "$OUT"
```

```
rg -n "\s*cloud\s*\{\}\bterraform\s*\{\s*$" "$ROOT" | tee -a "$OUT" || true
```

```
echo "" | tee -a "$OUT"
```

```
echo "== Dependency lock files ==" | tee -a "$OUT"
```

```
find "$ROOT" -name ".terraform.lock.hcl" -print | tee -a "$OUT" || true
```

```
echo "" | tee -a "$OUT"
```

```
echo "== Test files ==" | tee -a "$OUT"
```

```
find "$ROOT" -type f \( -name "*.tftest.hcl" -o -name "*.tftest.json" -o -name "*.tofutest.hcl" -o -name  
"*.tofutest.json" \) \
```

```
-print | tee -a "$OUT" || true
```

```
echo "" | tee -a "$OUT"
```

```
echo "== Policy-ish keywords (customize after discovery) ==" | tee -a "$OUT"
```

```
rg -n '(policy|url[_-]?list|allow[_-]?list|deny[_-]?list|block[_-]?list|siem|log\s*export)' "$ROOT" \
```

```
| tee -a "$OUT" || true
```

```
echo "" | tee -a "$OUT"
```

```
echo "Wrote: $OUT" | tee -a "$OUT"
```

Gap Analysis Template

Use the table below to compare "as-is" manual delivery vs the desired managed pipeline. The goal is to make gaps measurable and assignable.

Attribute	Current manual flow (fill in)	Key risks & failure modes	Desired managed pipeline behavior	Evidence/artifacts to store	Owner role label
Security			Least-privilege execution identity; protect state/plan (sensitive); OIDC/short-lived credentials where possible	Access logs; CI run logs; artifact retention	Platform Security

Attribute	Current manual flow (fill in)	Key risks & failure modes	Desired managed pipeline behavior	Evidence/artifacts to store	Owner role label
Testing			Manual validation misses edge cases; regressions	PR gates: fmt/validate; policy-as-code; automated tests; sandbox verification	Test reports; plan JSON policy results
Approvals			Untracked approvals; inconsistent signoff	Protected environments with required reviewers; merge protections	Approval logs; PR review history
Drift detection			Silent drift breaks policy guarantees	Scheduled drift checks (managed or CI); alert + ticket workflow	Drift reports; remediation issues
State management			Concurrent writes; corruption; wrong workspace	Remote backend + locking; one apply authority per scope	State location map; lock incidents
Secrets			Secret leakage via repos, plans, state exports	No secrets in repo; minimize plan/state exposure; restrict JSON outputs	Secret scans; vault usage logs
Rollback			No deterministic rollback path	“Revert commit + apply” playbook; state rollback process if using managed state versions	Rollback runbooks; incident notes
Observability			Poor auditability; missing change trace	Centralized logs; policy change audit trail; export logs to SIEM if available	CI artifacts; SIEM ingestion
Compliance			Missing evidence for audits	Policy-as-code gates; preserved plans/tests/approvals mapping to controls	Evidence bundles per release

Foundational constraints for these rows come from Terraform’s warning that sensitive values may land in plan/state, and from state locking and workspace isolation guidance.

Target Managed Pipeline Architecture

A robust architecture provides consistent governance regardless of which runner (OpenTofu or Terraform) you choose. The main design choice is *where* you want to sit on the spectrum between “VCS-native CI” and “specialized IaC automation platforms.”

CI/CD tooling options

Option	What it does	Strengths	Tradeoffs	When to choose
VCS-native CI (GitHub Actions / GitLab CI)	Run plan/test/scans on PR; apply on merge with environment protections	Maximum flexibility; native environment approvals and protections	You build drift & evidence plumbing yourself	You want tight integration, minimal extra services
Atlantis	PR-driven Terraform automation; autoplan; centralized apply	Standardizes PR workflow and keeps credentials off laptops	Self-host + secure additional service	You want PR comments + controlled execution
Managed runs (HashiCorp platform)	Centralized state, drift detection, policy enforcement	Built-in drift detection via health assessments; state version mgmt	Platform adoption/licensing; process coupling	You want turnkey drift detection & state governance

Branching strategy and promotion workflow

A pragmatic default is **trunk-based development with short-lived feature branches** and an “apply-after-merge” model: PRs generate plans and validations; applies run from the trusted main branch with environment gating. This reduces risk versus applying unmerged code and aligns with how PR automation tools commonly operate.

Promotion path (recommended):

- **PR (feature branch)**: fmt/validate → plan (sandbox target) → policy-as-code checks (OPA/Conftest) → security scan (Checkov plan scan) → review using plan output.
- **Merge to main**: apply to sandbox automatically; run post-apply verification (API readback + assertions); publish evidence bundle.
- **Production deploy job**: gated by protected environment approval (GitHub deployment protection rules / GitLab deployment approvals + protected environments).

Canary and gradual rollout

Because Island’s exact targeting model is unspecified, define canary in a way that matches how policies are assigned (e.g., pilot user group, pilot org, pilot device cohort). The pipeline should support:

- **Scoped apply** (apply policy change only to the canary target first).
- **Automated verification** against live telemetry or API reads.
- **Promotion** from canary → broader rollout once checks pass and approvals are obtained.

This is easiest if your IaC layer models *policy assignment* as a first-class object (resource) so you can deploy the policy definition once and gradually expand its assignments.

Workflow diagram (Mermaid)

mermaid

Copy

flowchart TB

PR[Pull/Merge Request: policy intent + IaC change] --> CI1[CI: fmt, validate]

CI1 --> PLAN[CI: plan (sandbox target) + plan JSON export]

PLAN --> PAC[Policy-as-code checks: OPA/Conftest + scanners]

PAC --> TESTS[IaC tests: terraform test / tofu test where applicable]

TESTS --> REVIEW[Review: PR approvals + plan review]

REVIEW --> MERGE[Merge to main]

MERGE --> SBX_APPLY[Apply to sandbox]

SBX_APPLY --> VERIFY[Post-apply verification: API readback + assertions]

VERIFY --> PROD_GATE[Prod gate: protected environment approvals]

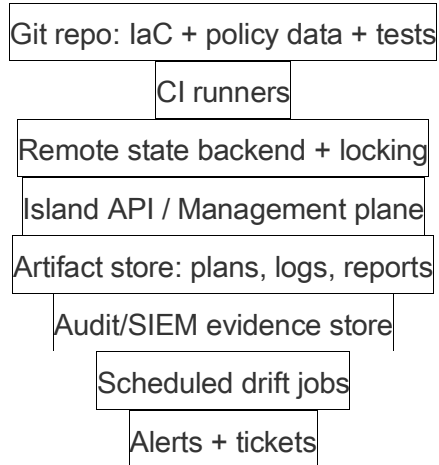
PROD_GATE --> PROD_APPLY[Apply to production]

PROD_APPLY --> EVIDENCE[Publish evidence bundle: logs, plan JSON, approvals]

EVIDENCE --> DRIFT[Scheduled drift checks + alerts]

Key architectural controls in this diagram are grounded in environment protections (approvals) and plan-as-artifact patterns described in GitHub/GitLab docs and OPA/Checkov plan evaluation guidance.

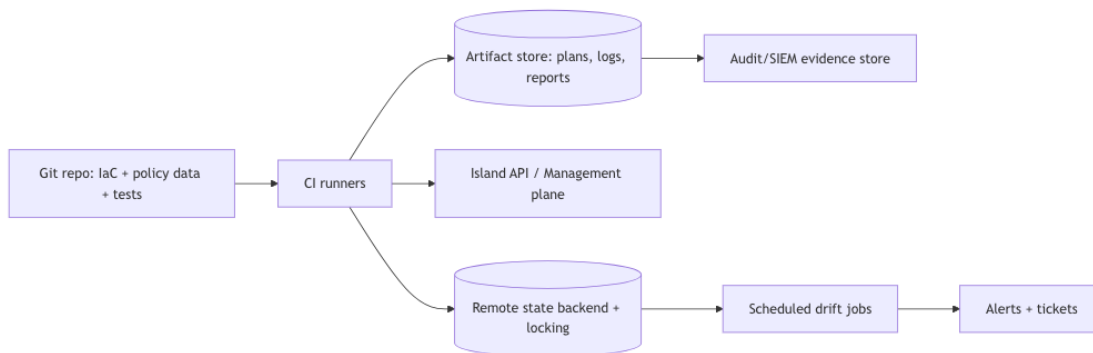
Entity relationship diagram (Mermaid)



Show code

The importance of state backend controls and locking is emphasized by Terraform's backend and state locking documentation.

Entity relationship diagram (Mermaid)



The importance of state backend controls and locking is emphasized by Terraform's backend and state locking documentation.

OpenTofu and Terraform Integration Patterns

This section provides concrete patterns for: coexistence, provider/plugin compatibility, state migration, wrapper scripts, and CI examples.

Provider vs “external program” patterns for Island

Preferred: Terraform/OpenTofu provider (first-class resource model).

Terraform’s model expects providers to define resources and data sources; each root module must declare required providers so the CLI can install and use them.

If an Island provider exists (internal or public), it’s usually the best long-term fit because it enables:

- Idempotent CRUD of policy objects (policy, URL lists, assignments).
- Drift detection aligned to provider reads.
- Cleaner state semantics and rollback via code reverts.

Fallback: “external program” escape hatch.

If no provider exists, you may integrate an external policy publisher program temporarily:

- The Terraform “external provider” exists to interface with external programs and is intended for simple cases where a first-class provider does not exist—explicitly framed as an exceptional mechanism (“escape hatch”) in the upstream project.
- Note: the external provider is primarily a `data source` mechanism; it does not inherently provide a robust resource lifecycle, so you often end up coupling it with other constructs and scripts. Treat this as a transitional solution and plan to replace it with a provider-backed implementation when feasible.

Provider source addressing and registry defaults

To avoid surprises during coexistence/migration, make provider source addresses explicit and commit lock files.

- Terraform defaults the `hostname/` portion of provider addresses to `registry.terraform.io/` when omitted.
- OpenTofu’s provider addressing uses `hostname/namespace/type` and the public OpenTofu registry is `registry.opentofu.org`; OpenTofu’s CLI config also describes how the “origin registry” is encoded in provider source address.
- Terraform recommends committing `.terraform.lock.hcl` for reviewable and reproducible provider selection across runs and platforms.

Example: explicit provider requirements (pattern, not Island-specific)

`hcl`
Copy

```
terraform {  
  required_providers {  
    island = {  
      source = "examplecorp/island"    }  
  }  
}
```

```
version = "~> 0.1"
```

Plugin protocol and compatibility considerations

Provider compatibility is mediated by the Terraform plugin protocol. HashiCorp documents protocol versions 5 and 6 and their CLI compatibility ranges.

OpenTofu's v1.x compatibility promises explicitly state it will support provider plugin protocol v5 throughout v1.x (and may add newer protocol versions while keeping v5).

Practical guidance:

- During a migration/cutover, **avoid coupling toolchain migration with provider upgrades**; pin provider versions and only change one "axis" at once (runner vs provider). Lock files and pinned versions support this discipline.

Coexistence strategies

Strategy A: single apply authority per scope (recommended).

Choose one runner (Terraform or OpenTofu) as the only engine allowed to **apply** per state scope; allow the other to run **fmt/validate/plan** only. This reduces risk of unintended state mutations and relies on state locking as a backstop, not as a primary coordination mechanism.

Strategy B: dual-plan parity, single-apply.

In CI, generate a Terraform plan and an OpenTofu plan from the same commit and require "no unexpected differences." OpenTofu migration guides emphasize inspecting the plan carefully before applying.

Strategy C: split domains (new OpenTofu stacks only).

New policy domains use OpenTofu with new state locations; legacy stays on Terraform until later. This reduces immediate migration risk but increases operational complexity and fragmentation (two "standards"). This pattern is most appropriate when you can clearly segment policy scopes.

State migration Terraform → OpenTofu (controlled cutover)

OpenTofu migration guidance consistently sequences:

- DR plan and ensure Terraform has no pending changes (**terraform plan** shows no changes),
- backup state and code,
- initialize OpenTofu, inspect plan, apply, then test a small change.

This sequence is explicitly designed to preserve rollback options through backups and to reduce complexity by migrating from a clean Terraform state.

Backend configuration and secrets hygiene

Backend configuration is a common leakage vector. OpenTofu warns against using variables in backend config to specify secrets because it can leak credentials if misconfigured; it also notes backend values must be resolvable during `tofu init` before state is available.

Terraform similarly emphasizes that state/plan can store sensitive values, and that sensitive data protection requires careful design.

CLI wrapper pattern (standardizing local and CI invocation)

A wrapper script helps enforce:

- correct working directory selection,
- consistent init flags,
- consistent logging and artifact paths,
- runner selection (Terraform vs OpenTofu) via config.

bash

Copy

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
# Usage:
```

```
# ./iac.sh plan environments/sandbox
```

```
# IAC_BIN=tofu ./iac.sh apply environments/prod
```

```
CMD="${1:?plan|apply|validate|fmt|test}"
```

```
DIR="${2:-.}"
```

```
IAC_BIN="${IAC_BIN:-terraform}" # switch to tofu during coexistence
```

```
COMMON_ARGS=(-chdir="$DIR")
```

```
case "$CMD" in
```

```
fmt)
```

```
"$IAC_BIN" fmt -recursive -check
```

```
;;
```

```
validate)
```

```
"$IAC_BIN" "${COMMON_ARGS[@]}" init -input=false
```

```
"$IAC_BIN" "${COMMON_ARGS[@]}" validate
```

```

..
plan)
"$IAC_BIN" "${COMMON_ARGS[@]}" init -input=false
"$IAC_BIN" "${COMMON_ARGS[@]}" plan -out=plan.bin
"$IAC_BIN" "${COMMON_ARGS[@]}" show -json plan.bin > plan.json
..
apply)
"$IAC_BIN" "${COMMON_ARGS[@]}" init -input=false
"$IAC_BIN" "${COMMON_ARGS[@]}" apply -auto-approve
..
test)
"$IAC_BIN" "${COMMON_ARGS[@]}" test
..
*)
echo "Unknown command: $CMD" >&2
exit 2
..
esac

```

Be cautious with plan/state JSON artifacts: OpenTofu warns `tofu show -json` returns sensitive values in plaintext, so treat `plan.json` as sensitive and restrict retention and access.

Sample CI YAML for GitHub Actions

This example demonstrates:

- PR: fmt/validate → plan → OPA/Conftest → Checkov plan scan
- merge: sandbox apply
- prod: gated by environment protections
- uses OIDC where possible to avoid long-lived secrets

yaml

Copy

```
name: island-policy-pipeline
```

```
on:
```

```
  pull_request:
```

push:

branches: ["main"]

schedule:

- cron: "0 3 * * *" *# drift check*

permissions:

contents: read

id-token: write *# needed for OIDC-based cloud auth patterns*

env:

IAC_BIN: tofu *# or terraform during coexistence*

SBX_DIR: environments/sandbox

PROD_DIR: environments/prod

jobs:

pr_checks:

if: github.event_name == 'pull_request'

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Format

run: `${{ env.IAC_BIN }} fmt -check -recursive`

- name: Validate

run: |

`${{ env.IAC_BIN }} -chdir=${{ env.SBX_DIR }} init -input=false`

`${{ env.IAC_BIN }} -chdir=${{ env.SBX_DIR }} validate`

- name: Plan (sandbox)

run: |

`${{ env.IAC_BIN }} -chdir=${{ env.SBX_DIR }} plan -out=plan.bin`

```
${{ env.IAC_BIN }} -chdir=${{ env.SBX_DIR }} show -json plan.bin > plan.json
```

```
- name: OPA/Conftest policy checks
```

```
run: conftest test plan.json -p policy/
```

```
- name: Checkov plan scan
```

```
run: checkov -f plan.json
```

```
apply_sandbox:
```

```
if: github.event_name == 'push'
```

```
runs-on: ubuntu-latest
```

```
steps:
```

```
- uses: actions/checkout@v4
```

```
- name: Authenticate (OIDC preferred)
```

```
run: echo "Configure cloud auth here (OIDC where supported)"
```

```
- name: Apply sandbox
```

```
run: |
```

```
  ${{ env.IAC_BIN }} -chdir=${{ env.SBX_DIR }} init -input=false
```

```
  ${{ env.IAC_BIN }} -chdir=${{ env.SBX_DIR }} apply -auto-approve
```

```
apply_prod:
```

```
if: github.event_name == 'push'
```

```
runs-on: ubuntu-latest
```

```
environment: production # protect with deployment protection rules
```

```
steps:
```

```
- uses: actions/checkout@v4
```

```
- name: Apply production
```

```
run: |
```

```
  ${{ env.IAC_BIN }} -chdir=${{ env.PROD_DIR }} init -input=false
```

```
{{ env.IAC_BIN }} -chdir={{ env.PROD_DIR }} apply -auto-approve
```

This structure aligns with:

- GitHub environment deployment protection rules (manual approvals / restrictions).
- GitHub OIDC guidance (avoid long-lived secrets).
- OPA plan-to-JSON evaluation workflow (terraform show -json).
- Checkov's official Terraform plan scanning workflow.

Sample CI YAML for GitLab

yaml

Copy

```
stages: [check, plan, deploy]
```

```
variables:
```

```
IAC_BIN: "tofu"
```

```
SBX_DIR: "environments/sandbox"
```

```
PROD_DIR: "environments/prod"
```

```
fmt_validate:
```

```
stage: check
```

```
script:
```

```
- $IAC_BIN fmt -check -recursive
```

```
- $IAC_BIN -chdir=$SBX_DIR init -input=false
```

```
- $IAC_BIN -chdir=$SBX_DIR validate
```

```
plan_sandbox:
```

```
stage: plan
```

```
script:
```

```
- $IAC_BIN -chdir=$SBX_DIR plan -out=plan.bin
```

```
- $IAC_BIN -chdir=$SBX_DIR show -json plan.bin > plan.json
```

```
- conftest test plan.json -p policy/
```

```
- checkov -f plan.json
```

```
artifacts:
```

paths:

- plan.json

deploy_prod:

stage: deploy

environment:

name: production # protect + require deployment approvals

script:

- \$IAC_BIN -chdir=\$PROD_DIR init -input=false

- \$IAC_BIN -chdir=\$PROD_DIR apply -auto-approve

rules:

- if: '\$CI_COMMIT_BRANCH == "main"'

GitLab's deployment approvals and protected environments block deployments until approvals are granted and restrict who can deploy.

Testing, Validation, Observability, and Drift Detection

Baseline gates for every PR

- **Formatting and static validation:** `terraform fmt` formats code canonically, and `terraform validate` checks configuration validity without validating remote services.
- **Unit/integration tests:** `terraform test` can create real infrastructure and therefore can cost money; the Terraform language testing docs note tests are integration-like by default (real infrastructure), so decide what's safe to run per PR vs nightly.
- **Policy-as-code (OPA + Conftest):** OPA's Terraform guidance uses `terraform show -json` to convert a plan into JSON that OPA can evaluate, and Conftest describes itself as a utility for testing structured configuration data using Rego.
- **Plan scanning (Checkov):** Checkov's own docs provide an explicit "plan scanning" example: create a binary plan, export to JSON, and run `checkov -f tfplan.json`.

Testing tactics tailored to "policy updates" (not infrastructure)

Because your actual payload is "policy configuration," you can often test without provisioning infrastructure by focusing on **plan diff correctness** and **API readback**:

- **Contract tests against the Island API schema:** If you have an OpenAPI spec (the marketplace listing claims OpenAPI/Swagger docs exist), validate policy payloads against

the schema as part of CI. This reduces runtime failures and guards against breaking API changes.

- **Policy intent unit tests:** Encode invariants (e.g., "every URL list entry includes an owner + justification + expiration," "no wildcard domains unless explicitly approved," "deny rules require ticket reference") in OPA/Rego + Conftest tests. OPA supports policy testing frameworks to validate the correctness of policies themselves.
- **Post-apply verification:** Use Terraform/OpenTofu "check-like" validations or external readback scripts that confirm the Island policy state matches expected values and that logs/telemetry are flowing (where available). Terraform's validation features explicitly support documenting assumptions and validating infrastructure behavior via checks and pre/postconditions.

Observability and evidence

Minimum evidence bundle per production change:

- Plan output (human-readable) and plan JSON (restricted access due to sensitivity).
- CI logs, approval records, and policy-as-code results.
- Post-apply verification outputs.
- A change record that links PR → environment deploy → policy targets.

This is justified by the sensitivity of state/plan files and by the governance requirements implied by environment deployment protection rules and auditability.

Drift detection options

Managed drift detection (if adopting HashiCorp managed runs).

HCP Terraform health assessments provide drift detection; the workspace health docs describe where drift results are viewed and that drift detection proposes changes to bring infra back in sync.

CI-native drift detection (Terraform/OpenTofu scheduled jobs).

- Use scheduled "refresh-only" or drift-focused plans plus alerting. OpenTofu's machine-readable UI emits `resource_drift` messages when drift is detected during planning, which you can parse to produce actionable alerts.
- Be careful with OpenTofu's deprecated `tofu refresh` command; OpenTofu documentation marks it deprecated and recommends safer alternatives (plan-based).

Migration Checklist and Risk Mitigation Plan

Migration checklist (manual → managed pipeline)

1. Define the unit of delivery and state scope.

Decide what maps to one state: per environment (sandbox/prod), per tenant/org, or per policy domain. Avoid using workspaces as your security boundary if separate credentials/access are required.

2. **Secure and standardize state backends.**
Backends store state and may provide locking; locking prevents concurrent writers. Enable versioning where applicable (e.g., S3 backend recommends bucket versioning).
3. **Commit lock files and pin providers.**
Commit `.terraform.lock.hcl` and explicitly manage provider versions to prevent surprise upgrades.
4. **Introduce PR plan checks before automating apply.**
Start with `fmt/validate + plan + policy-as-code + scan` gates; do not change production deployment mechanics yet.
5. **Automate sandbox apply with verification.**
After merge, apply to sandbox and validate via API readback and assertions.
6. **Add production gating via protected environments.**
Use GitHub/GitLab environment protection rules and deployment approvals to enforce signoff in the system, not via side-channel.
7. **Add drift checks + alerting.**
Choose managed health assessments or CI-native scheduled drift checks; integrate with tickets/on-call.
8. **If adopting OpenTofu: migrate carefully with backups and clean state.**
Follow OpenTofu's migration guides: DR plan, apply pending changes, backup state/code, then `init/plan/apply/test`.

Risk mitigation table

Risk	Why it matters	Mitigations grounded in docs/practice
Secret exposure via plan/state artifacts	Terraform states secrets can land in state/plan files; OpenTofu warns JSON output reveals sensitive values in plaintext	Restrict artifact access/retention; minimize JSON exports; use OIDC; apply least privilege
State corruption / parallel applies	Terraform locks state for operations that could write state; disabling locking is not recommended	Ensure one apply authority per scope; enforce locking; document break-glass procedures
Misuse of workspaces for environment isolation	Terraform warns workspaces are not appropriate for deployments requiring separate credentials/access controls	Prefer separate backends per env/tenant; avoid workspace-as-security-boundary
Runner migration instability (Terraform ↔ OpenTofu)	OpenTofu migration guides emphasize DR, backups, plan inspection prior to apply	Dual-plan parity; staged cutover; keep rollback via backups
Provider incompatibility	Provider protocol versions affect compatibility; OpenTofu promises v5 support across v1.x	Pin providers; don't couple runner migration with provider upgrades
Drift and silent divergence	Drift breaks policy guarantees and compliance assertions	Health assessments (managed) or scheduled drift checks; parse drift events

Prioritized Action Plan and References

Prioritized action plan (milestones, effort, owners)

Milestone	Deliverables	Effort	Owner role labels
Discovery baseline	Current-state runbook; artifact inventory; state/backends map; manual controls & acceptance criteria mapped	Medium	IaC Owner; Policy SME; Platform Eng
PR governance MVP	PR checks: fmt/validate + plan + OPA/Conftest + Checkov plan scan; evidence artifacts policy	Medium	IaC Owner; Platform Security
Sandbox automation	Merge-to-main triggers sandbox apply; post-apply verification; evidence bundle stored	Medium	IaC Owner; SRE
Production gating	Protected environment approvals; prod apply job; rollback playbook (“revert commit + apply”)	Medium	Change Mgmt; Platform Eng
Drift detection	Scheduled drift checks (managed or CI-native); alert routing; remediation SOP	Medium	SRE; Observability
OpenTofu/Terraform strategy	Decision doc: coexistence rules or migration plan; version pinning and provider registry approach	Medium	IaC Owner; Platform Eng; Security
Provider strategy for Island	Confirm provider existence; if none, define build vs interim external-program approach; roadmap	High	IaC Owner; Platform Eng; Policy SME
Full cutover	Manual steps retired; training; audit mapping; continuous improvement loop for policies/tests	High	IaC Owner; Compliance; SRE

Key authoritative sources and links

Citations throughout this report point to primary documentation for Terraform/OpenTofu, CI platforms, and policy-as-code/scanning tooling. You requested explicit links; below are direct URLs in a code block (so they remain copyable).

text

Copy

Terraform (HashiCorp Developer):

- <https://developer.hashicorp.com/terraform/language/manage-sensitive-data>
- <https://developer.hashicorp.com/terraform/language/state/backends>
- <https://developer.hashicorp.com/terraform/language/state/locking>
- <https://developer.hashicorp.com/terraform/language/state/workspaces>
- <https://developer.hashicorp.com/terraform/language/files/dependency-lock>
- <https://developer.hashicorp.com/terraform/internals/provider-registry-protocol>
- <https://developer.hashicorp.com/terraform/plugin/terraform-plugin-protocol>
- <https://developer.hashicorp.com/terraform/cli/commands/test>
- <https://developer.hashicorp.com/terraform/cli/commands/fmt>
- <https://developer.hashicorp.com/terraform/cli/commands/validate>
- <https://developer.hashicorp.com/terraform/cloud-docs/workspaces/health>
- <https://developer.hashicorp.com/terraform/cloud-docs/workspaces/state>

OpenTofu:

- <https://opentofu.org/docs/intro/migration/migration-guide/> (and version-specific migration guides)

- <https://opentofu.org/docs/cli/commands/init/>
- <https://opentofu.org/docs/cli/commands/plan/>
- <https://opentofu.org/docs/cli/commands/show/>
- <https://opentofu.org/docs/cli/commands/test/>
- <https://opentofu.org/docs/cli/config/config-file/>
- <https://opentofu.org/docs/language/v1-compatibility-promises/>
- <https://opentofu.org/docs/internals/machine-readable-ui/>
- <https://opentofu.org/docs/language/state/encryption/>
- <https://opentofu.org/docs/internals/provider-registry-protocol/>

GitHub Actions:

- <https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>
- <https://docs.github.com/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect>
- <https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-cloud-providers>

GitLab:

- https://docs.gitlab.com/ci/environments/protected_environments/
- https://docs.gitlab.com/ci/environments/deployment_approvals/

OPA / Conftest:

- <https://openpolicyagent.org/docs/terraform>
- <https://openpolicyagent.org/docs/policy-testing>
- <https://www.conftest.dev/>
- <https://github.com/open-policy-agent/conftest>

Checkov:

- <https://www.checkov.io/7.Scan%20Examples/Terraform%20Plan%20Scanning.html>
- <https://github.com/bridgecrewio/checkov>

Atlantis:

- <https://www.runatlantis.io/docs/autopanning>

- <https://github.com/runatlantis/atlantis>

Terraform external provider:

- <https://registry.terraform.io/providers/hashicorp/external/latest/docs>

- <https://github.com/hashicorp/terraform-provider-external>

Notable platform capability pointers used in this report

- GitHub environment deployment protection rules (approvals, delays, branch restrictions).
- GitHub OIDC for cloud access without long-lived secrets.
- GitLab deployment approvals and protected environments.
- OPA workflow for Terraform plan JSON evaluation.
- Conftest definition as tests for structured config with Rego.
- Checkov Terraform plan scanning example.
- OpenTofu migration practices and compatibility promises.
- Island API claims about policy CRUD and URL lists (public procurement listing).